

JT_DEV_B41_2.7

Web todo list with TypeScript

1 Initial situation

- 1.1 Nowadays, we mainly create applications with a graphical interface. Web applications are very popular because they are easy to access and do not require installation. In this project, you will create a simple to-do list application with only HTML, CSS and TypeScript.
- 1.2 The application is relatively simple, offering an interface for adding to-dos, marking them as complete and deleting them. You can also give them an expiry date.
We also want some display functions, such as filters to display only to-dos that have already been validated or that still need to be done. Or display only to-dos that have expired.
- 1.3 As usual, you will manage your codebase with git and use conventional commits. To start the project, your trainer will provide you with a canvas containing the initial structure for creating a typescript web application.
For this project, you will work with a git workflow [inspired by trunk-based development](#). For each feature, you will create a git branch and a pull request.

2 Skills from the ICT training plan worked on

Skills ranked by importance of implementation in the project: [consult the training plan](#).

- 2.1 g5 : Implémenter les applications et les interfaces selon le concept en respectant les exigences de sécurité.
- 2.2 a3 : Rechercher des informations sur des solutions ICT et sur les innovations.
- 2.3 a6 : Vérifier l'avancement des projets ICT et des tâches et en faire état selon le modèle de procédure.
- 2.4 g2 : Vérifier la faisabilité technique des prototypes d'interfaces utilisateur et les développer.
- 2.5 g6 : Vérifier la qualité et la sécurité des applications et des interfaces.
- 2.6 a7 : Remettre la solution ICT au client et clôturer le projet.
- 2.7 h3 : Implémenter le processus de livraison des applications.

3 Deliverables

- 3.1 Github repository containing your project, it must work out of the box (following the setup guide provided in your README).
- 3.2 A working Github pages deployment of your project.
- 3.3 Your GitHub project board must be completed (all tasks closed by a [keyword](#)).

4 Andragogic conditions

- 4.1 Work is carried out independently, on an individual basis
- 4.2 You are allowed to help each other, share ideas and discuss solutions, but each trainee must provide their own implementation with their own code.
- 4.3 Apprentices have access to all the resources they need, particularly on the web.
- 4.4 Don't hesitate to ask AI for explanations, but avoid asking it for answers. The more you ask for answers, the less you learn.



- 4.5 Maximum allowed time for the project: 13 full days (7.5 hours per day).** You must manage your time independently. On project beginning, inform your trainer of the start date and time.

5 Steps

- 5.1** Create a new repository on your personal account **from the following [template repository](#)** (use the green « Use this template » button).
- 5.2** Create a new GitHub Project from [the web todo template](#) to your account (Use the « ... » then « Make a copy » menu). Select your account to host the Project on and name the project with your GitHub pseudo, for example `web-todo-<your-username>`. **Do not forget to tick the « Draft issues will be copied if selected » checkbox!**
- 5.3** Link your GitHub project to your GitHub repository (« Projects » tab on your repository page).
- 5.4** Convert all the draft issues to plain issues linked to your newly created GitHub repository.
- 5.5** Give your trainer (GitHub pseudo `fuzoh`) read access to **your project and to your repository**.
- 5.6** Activate Gemini code assist on your GitHub account: <https://github.com/marketplace/gemini-code-assist> (this AI will automatically review your code on each pull requests and give you suggestions to improve it).
- 5.7** Complete all the tasks from the GitHub project. **For each task, take time to read them attentively, before merging your code check that all acceptance criteria are met.**

6 Resources

- 6.1** https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- 6.2** <https://developer.mozilla.org/en-US/docs/Glossary/DOM>
- 6.3** <https://grafikart.fr/formations/typescript>
- 6.4** Documentation for all available endpoints in the todolist RESTful API: <https://swagger.todos.in.jt-lab.ch/>
- 6.5** Documentation to make specific queries to the RESTful API: https://docs.postgrest.org/en/v12/references/api/tables_views.html
- 6.6** Some good typescript practices: <https://www.typescriptlang.org/docs/handbook/declaration-files/do-s-and-don-ts.html>
- 6.7** What is a user story: <https://www.atlassian.com/agile/project-management/user-stories>
- 6.8** Reminders for git usage: <https://www.figma.com/board/NZjAxUSIE0EiSv9uavCoKK/Utilisation-de-Git?node-id=0-1&t=vDTVZQMDjoi1CHqi-1>

7 Evaluation criteria

- 7.1** The project is deployed with GitHub pages and works out of the box.
- 7.2** All the requested features must be implemented and working. **The specified html tags and ids must be used.**
- 7.3** All commit messages must comply with [conventional commits](#) standard.
- 7.4** All the codebase must be formatted according to the template repository configuration (biome) `pnpm format`.
- 7.5** There are no lints errors (with biome) `pnpm lint`.
- 7.6** There are no type errors (with tsc) `pnpm check-types`.
- 7.7** There is one pull request and one branch per task of the project (except the first one). The pull request contains the corresponding [close keyword](#).
- 7.8** There is **no code repetitions** in the codebase.
- 7.9** The README must contain instructions to launch the project.



- 7.10 Following the README instructions to launch the project must work out of the box.
- 7.11 As always, the README should contain a brief description of the app and a list of features.
- 7.12 The style of the page must look simple and modern. Using latest CSS features.
- 7.13 The project respects TypeScript and web best practices.
- 7.14 There is no dead code.
- 7.15 No external libraries are used (no CSS or JS libraries).
- 7.16 At the end, a technical presentation of the codebase will be made to your trainer.

